

Week 2 — Mathematical Foundations

The four maths tools a network runs on: shapes, dot products, gradients, the chain rule.

1. Vectors, Matrices, Tensors

- Vector** — an ordered list of numbers. This is a **1st-order tensor**. n numbers $\rightarrow$ shape $(n,)$.
- Matrix** — a 2D grid of numbers. This is a **2nd-order tensor**. Shape $(m \times n)$ = rows $\times$ columns.
- Tensor** — the general word for an array of numbers with any number of dimensions. 3 or more dimensions, for example an RGB image: height $\times$ width $\times$ colour channels.

In a network	What it is
One training example	A vector
The weights of one layer	A matrix
An RGB image	A tensor

- PyTorch and TensorFlow store **all** of these as one single type, called a “tensor”.

2. Shapes and Matrix Multiplication

The Matrix Multiplication Rule

$$A \in \mathbb{R}^{m \times n}, B \in \mathbb{R}^{n \times p} \implies AB \in \mathbb{R}^{m \times p}$$

- The **inner** sizes (the two n values that touch) **must be equal**.
- The answer takes the two **outer** sizes: m from the left, p from the right.
- If the inner sizes are different, the multiplication has no meaning and the code stops with an error.
- Exam note:** a wrong shape is the **single most common bug** when writing neural network code.

The Dot Product

Dot Product — two ways to write the same thing

$$a \cdot b = \sum_{i=1}^n a_i b_i, \quad a \cdot b = \|a\| \|b\| \cos \theta$$

$\|a\|$ = the length of vector a ; θ = the angle between the two vectors.

Angle between them	Dot product	What it means
0° — same direction	Large positive	Very similar
180° — opposite direction	Large negative	Opposite
90° — at right angles	Exactly 0	Not similar at all

- This is the idea behind **cosine similarity**. The dot product measures **how similar the directions of two vectors are**, whatever their lengths.
- Why this matters:** every neuron takes the dot product of the input with its own weight vector. So the neuron is measuring **how closely the input matches the pattern stored in its weights**. The bias then moves the point at which the neuron starts to turn on.

3. A Whole Layer in One Multiplication

Full Layer Forward Computation

$$z = Wx + b$$

- $W \in \mathbb{R}^{m \times n}$: m = number of neurons in the layer, n = number of input features. **One row of W per neuron.**
- $x \in \mathbb{R}^n$ = the input vector. $b \in \mathbb{R}^m$ = one bias per neuron. $z \in \mathbb{R}^m$ = one raw output per neuron.
- For a batch of examples:** put each input vector as one **column** of a matrix X . Then the single multiplication WX computes the answer for **every example at the same time**.
- This is exactly why GPUs make deep learning fast. A GPU does many multiplications side by side, and WX is built out of exactly that kind of work.

4. Derivatives and Gradients

- Derivative** — how much the output changes when you change the input by a tiny amount. It is the **slope** of the function at one point.

$$\frac{dy}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- Partial derivative** — the same idea, but you change **one** variable and keep every other variable fixed. It is like turning one knob at a time.

- Gradient** — one vector that holds **all** the partial derivatives together.

Gradient Vector

$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$$

It has the **same size as the input** — one number per input, saying how much that input changes the output.

What the gradient means, in pictures:

- It points in the direction where the function **goes up fastest**.
- Its size says **how fast** the function changes there: large = steep, near zero = almost flat.
- $\nabla f = 0 \rightarrow$ a **flat spot**. This can be a minimum, a maximum, or a **saddle point** (a place that goes up in one direction and down in another). So a zero gradient does **not** prove you are at the bottom.
- It always sits at right angles to the lines where the function keeps the same value.

Why learning uses the MINUS sign

Training must make the loss **smaller**. The gradient points the way the loss gets **bigger**. So every step moves along $-\nabla f$, which is the direction where the loss falls fastest.

5. The Chain Rule

Chain Rule

If y depends on x **only** through a middle value u , that is $u = g(x)$ and $y = f(u)$:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

The total effect = **the product of the small effects** along the chain.

- In a deep network, the loss can reach a weight in an **early** layer only by passing through **every layer after it**.
- So the gradient for that early weight is the **product** of the small derivatives of every layer after it.
- Using this rule again and again, one layer at a time, **is** backpropagation. (*Full algorithm in Week 5.*)
- This also explains the danger:** multiply many small numbers $\rightarrow$ the result shrinks to almost nothing (vanishing gradient). Multiply many large numbers $\rightarrow$ it grows out of control (exploding gradient).

6. Numerical Stability

Problem	What happens	Damage
Overflow	The number is too big for the computer to store, so it is saved as ∞	Every later calculation that uses it becomes meaningless
Underflow	The number is too small, so it is rounded to exactly 0	The gradient or probability information is lost for good

Deep learning hits these often because of three things:

- Very long chains of multiplications.
- Heavy use of the exponential function e^x , which grows extremely fast.
- Probabilities that become extremely small.

The Log-Sum-Exp Trick

$$\log \sum_i e^{x_i} = c + \log \sum_i e^{x_i - c}, \quad c = \max_i x_i$$

After you take away the **largest** value c , the biggest term becomes $e^0 = 1$. Every other term then sits between 0 and 1. **Nothing can overflow, and nothing is lost**. Softmax and cross-entropy do this for you automatically.

Other ways to keep numbers safe:

- Work with the logarithms of the numbers instead of the numbers themselves.
- Take away the largest value before using e^x .
- Add a tiny number ε to the bottom of a fraction, so you never divide by zero.
- Replace a long multiplication of small probabilities with a sum of their logarithms.

MUST REMEMBER

- Vector = 1st-order tensor; matrix = 2nd-order tensor; tensor = an array with any number of dimensions.
- Matrix multiplication needs the **inner sizes to match**; the answer takes the **outer two**. A wrong shape is the number-one coding bug.
- $a \cdot b = \sum a_i b_i = \|a\| \|b\| \cos \theta$ — positive if the vectors point the same way, negative if opposite, **exactly zero if at right angles**.
- One neuron computes $w^T x + b$; a whole layer computes $Wx + b$ for every neuron at once; a batch stacks inputs so the GPU does them all together.
- Derivative = how much the output changes for a tiny change in the input. Partial derivative = the same, with all other variables held fixed.
- Gradient ∇f = all the partial derivatives in one vector. It points the way the function **rises** fastest. Learning moves along $-\nabla f$. A zero gradient is a flat spot — minimum, maximum **or saddle**.
- Chain rule $\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}$ — needed whenever a variable reaches the output only through a middle value. This is the **backbone of backpropagation**.
- Overflow $\rightarrow \infty$; underflow $\rightarrow 0$. The **log-sum-exp trick** (take away the largest value before using e^x) keeps softmax and cross-entropy safe.

LIKELY EXAM QUESTIONS (5+5 / 6+4 style)

- Why do shape mismatches happen? Why do they matter? $\rightarrow$ inner sizes must match; the number-one coding bug
- What does the sign of a dot product tell you? $\rightarrow$ positive = aligned, negative = opposite, zero = at right angles
- Why does learning use $-\nabla f$ and not $+\nabla f$? $\rightarrow$ gradient points uphill; loss must go down
- What does a zero gradient mean? Is it always a minimum? $\rightarrow$ flat spot: min, max or saddle
- Why is the chain rule needed in a deep network? $\rightarrow$ loss reaches early weights only through later layers
- What are overflow and underflow? How do we avoid them? $\rightarrow \infty$ and 0; log-sum-exp, subtract the max
- Why does one matrix multiply handle a whole batch? $\rightarrow$ stack inputs as columns $\rightarrow$ GPU does it at once